
第六课 二分查找与二叉查找树

林沐

内容概述

1.5道经典二分搜索与二叉查找(排序)树的相关题目

预备知识:二分查找基础知识

例1:插入位置(easy) (二分查找)

例2:区间查找(medium) (二分查找)

例3:旋转数组查找(medium) (二分查找)

预备知识:二叉查找(排序)树基础知识

例4:二叉查找树编码与解码(medium)

例5:逆序数(hard) (二叉查找树应用)

2.详细讲解题目解题方法、代码实现

预备知识:二分查找

已知一个**排序数组**A, 如 $A = [-1, 2, 5, 20, 90, 100, 207, 800]$,
另外一个**乱序数组**B, 如 $B = [50, 90, 3, -1, 207, 80]$,
求B中的任意某个元素, 是否在A中出现, **结果**存储在数组C中, **出现用1代表, 未出现用0代表**, 如, $C = [0, 1, 0, 1, 1, 0]$ 。

```
//返回结果数组                                //排序数组  
std::vector<int> search_array(std::vector<int> &sort_array,  
                               std::vector<int> &random_array) {  
                                //乱序数组  
}
```

思考, 最暴力的方法**复杂度**是多少? 有没有**更快**的方法?

预备知识:二分查找算法

二分查找 又称 **折半查找**，首先，假设表中元素是按 **升序排列**，将表 **中间位置** 的关键字与查找关键字比较：

1. 如果两者 **相等**，则 **查找成功**；
2. 否则利用 **中间位置** 将表分成 **前、后** 两个子表：
 - 1) 如果中间位置的关键字 **大于** 查找关键字，则进一步查找 **前一子表**
 - 2) 否则进一步查找 **后一子表**

重复以上过程，**直到** 找到满足条件的记录，使查找成功，或直到子表不存在为止，此时查找不成功。

例如，待搜索数字 $target == 2, 200$

数组 $A = [-1, 2, 5, 20, 90, 100, 207, 800]$

预备知识:二分查找算法

待查找值 $target = 2$

下标: [0, 1, 2, 3, 4, 5, 6, 7]

nums: [-1, 2, 5, 20, 90, 100, 207, 800]

第1次搜索:

搜索区间:[0, 7] [-1, 2, 5, 20, 90, 100, 207, 800]

搜索 $target = 2$ 小于 $nums[mid](mid == 3) = 20$:

第2次搜索:

搜索区间:[0, 2] [-1, 2, 5]

搜索 $target = 2$ 等于 $nums[mid](mid == 1) = 2$:

故找到!

预备知识:二分查找算法

待查找值 target = 200

下标: [0, 1, 2, 3, 4, 5, 6, 7]

nums: [-1, 2, 5, 20, 90, 100, 207, 800]

第1次搜索:

搜索区间[0, 7] [-1, 2, 5, 20, 90, 100, 207, 800]

搜索target = 200 大于 nums[mid](mid==3) = 20:

第2次搜索:

搜索区间: [4, 7] [90, 100, 207, 800]

搜索target = 200 大于 nums[mid](mid==5) = 100

第3次搜索:

搜索区间: [6, 7] [207, 800]

搜索target = 200 小于 nums[mid](mid==6) = 207

搜索区间: [6, 5]

该区间是一个非法区间, 该表不存在, 故target 不在数组中!

预备知识:二分查找(递归), 课堂练习

```
#include <vector>

//排序数组
bool binary_search(std::vector<int> &sort_array,
//搜索到返回true      int begin, int end, int target){
//否则返回false      //待搜索的区间左端、右端      //搜索目标
    if ( 1 ) {
        return false;
    }
    int mid = 2
    if (target == sort_array[mid]) { //当找到时
        3
    }
    else if ( 4 ) { //??时候找左区间
        return binary_search(sort_array, begin, mid-1, target);
    }
    else if ( 5 ) { //??时候找右区间
        return binary_search(sort_array, mid + 1, end, target);
    }
}
```

3分钟填写代码,
有问题随时提出!

预备知识:二分查找(递归), 实现

```
#include <vector>

//排序数组
bool binary_search(std::vector<int> &sort_array,
//搜索到返回true      int begin, int end, int target){
//否则返回false      //待搜索的区间左端、右端      //搜索目标
    if (begin > end) { //区间不存在！
        return false;
    }
    int mid = (begin + end) / 2 //找中点
    if (target == sort_array[mid]){ //当找到时
        return true;
    }
    else if (target < sort_array[mid]) { //??时候找左区间
        return binary_search(sort_array, begin, mid-1, target);
    }
    else if (target > sort_array[mid]) { //??时候找右区间
        return binary_search(sort_array, mid + 1, end, target);
    }
}
```

例如, 待搜索数字target == 2, 200

数组 A = [-1, 2, 5, 20, 90, 100, 207, 800]

预备知识:二分查找(循环), 课堂练习

//排序数组

//搜索目标

```
bool binary_search(std::vector<int> &sort_array, int target){  
    int begin = 0;  
    int end = sort_array.size() - 1;  
    while (1){  
        int mid = (begin + end) / 2;  
        if (target == sort_array[mid]){  
            2  
        }  
        else if (target < sort_array[mid]){  
            3  
        }  
        else if (target > sort_array[mid]){  
            4  
        }  
    }  
    5  
}
```

3分钟填写代码,
有问题随时提出!

预备知识:二分查找(循环), 实现

```
//排序数组      //搜索目标
bool binary_search(std::vector<int> &sort_array, int target){
    int begin = 0;
    int end = sort_array.size() - 1;
    while( begin <= end ){
        int mid = (begin + end) / 2;
        if (target == sort_array[mid]){
            return true;
        }
        else if (target < sort_array[mid]){
            end = mid - 1;
        }
        else if (target > sort_array[mid]){
            begin = mid + 1;
        }
    }
    return false;
}
```

例如, 待搜索数字target == 2, 200

数组 A = [-1, 2, 5, 20, 90, 100, 207, 800]

```

std::vector<int> search_array(std::vector<int> &sort_array,
                             std::vector<int> &random_array) {
    std::vector<int> result;
    for (int i = 0; i < random_array.size(); i++){
        int res = binary_search(sort_array,
                                0,
                                sort_array.size() - 1,
                                random_array[i]);

        result.push_back(res);
    }
    return result;
}

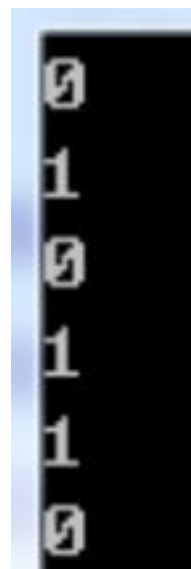
```

预备知识:测试

```

int main() {
    int A[] = {-1, 2, 5, 20, 90, 100, 207, 800};
    int B[] = {50, 90, 3, -1, 207, 80};
    std::vector<int> sort_array;
    std::vector<int> random_array;
    std::vector<int> C;
    for (int i = 0; i < 8; i++){
        sort_array.push_back(A[i]);
    }
    for (int i = 0; i < 6; i++){
        random_array.push_back(B[i]);
    }
    C = search_array(sort_array, random_array);
    for (int i = 0; i < C.size(); i++){
        printf("%d\n", C[i]);
    }
    return 0;
}

```



例1:插入位置

给定一个**排序数组nums(无重复元素)**与**目标值target**，如果target在nums里**出现**，则返回**target所在下标**，如果target在nums里**未出现**，则返回target**应该插入位置**的数组下标，使得将target插入数组nums后，数组仍**有序**。

```
class Solution {  
public:  
    int searchInsert(std::vector<int>& nums, int target) {  
    }  
};
```

	0 -> 0	4 -> 2
位置:[0, 1, 2, 3]	1 -> 0	5 -> 2
元素:[1, 3, 5, 6]	2 -> 1	6 -> 3
	3 -> 1	7 -> 4

选自 **LeetCode 35. Search Insert Position**

<https://leetcode.com/problems/search-insert-position/description/>

难度:**Easy**

例1:思考

1.当target在nums中**出现**时，二分查找的流程无变化。

2.当target在nums**没有出现**时:

1)如果 $\text{target} < \text{nums}[\text{mid}]$ ，且 $\text{target} > \text{nums}[\text{mid} - 1]$ ；

说明了什么？

2)如果 $\text{target} > \text{nums}[\text{mid}]$ ，且 $\text{target} < \text{nums}[\text{mid} + 1]$ ；

说明了什么？

3.当 $\text{mid} == 0$ 或者 $\text{mid} == \text{nums.size()} - 1$ 时，这样的**边界条件**，应该如何处理？

位置:[0, 1, 2, 3]

元素:[1, 3, 5, 6]

target = 2、4时，

$$\text{mid} = (0 + 3) / 2 = 1$$

$$\text{nums}[\text{mid}] = 3$$

$$\text{nums}[\text{mid} - 1] = 1$$

$$\text{nums}[\text{mid} + 1] = 5$$

例1:算法思路

设元素**所在的位置**(或最终需要**插入的位置**)为index,

在二分查找的**过程**中:

如果 $\text{target} == \text{nums}[\text{mid}]$: $\text{index} = \text{mid}$;

如果 $\text{target} < \text{nums}[\text{mid}]$, 且 $(\text{mid} == 0 \text{ 或 } \text{target} > \text{nums}[\text{mid}-1])$:
 $\text{index} = \text{mid}$;

如果 $\text{target} > \text{nums}[\text{mid}]$, 且 $(\text{mid} == \text{nums.size()} - 1 \text{ 或 } \text{target} < \text{nums}[\text{mid}+1])$:
 $\text{index} = \text{mid} + 1$;

位置:[0, 1, 2, 3]

元素:[1, 3, 5, 6]

target = 2、4时,

$\text{mid} = (0 + 3) / 2 = 1$

$\text{nums}[\text{mid}] = 3$

$\text{nums}[\text{mid} - 1] = 1$

$\text{nums}[\text{mid} + 1] = 5$

例1:算法思路

待查找值 target = 200

下标: [0, 1, 2, 3, 4, 5, 6, 7]

nums: [-1, 2, 5, 20, 90, 100, 207, 800]

第1次搜索:

搜索区间[0, 7] [-1, 2, 5, 20, 90, 100, 207, 800]

搜索target = 200 大于 nums[mid](mid==3) = 20:

并且target > nums[mid+1](90)

第2次搜索:

搜索区间: [4, 7] [90, 100, 207, 800]

搜索target = 200 大于 nums[mid](mid==5) = 100

并且target < nums[mid+1](207)

故 target 应该插入在6号位置!

例1:课堂练习

```
class Solution {
public:
    int searchInsert(std::vector<int>& nums, int target) {
        int index = -1; //最终返回的下标, 若找到则为该元素下标, 否则为需要插入的位置
        int begin = 0; //搜索区间左端点
        int end = nums.size() - 1; //搜索区间右端点
        while (index == -1) { //若index == -1, 则说明还未找到正确位置, 持续二分搜索
            int mid = (begin + end) / 2; //计算中心位置
            if (target == nums[mid]) { //若找到target
                

1


            }
            else if (target < nums[mid]) {
                if (
                    

2



3


                ) {
                }
                end = mid - 1; //如果target小于中点, 更新区间右端点
            }
            else if (target > nums[mid]) {
                if (
                    

4



5


                ) {
                }
                begin = mid + 1; //如果target大于终点, 更新区间左端点
            }
        }
        return index;
    }
};
```

位置:[0, 1, 2, 3]

元素:[1, 3, 5, 6]

target = 2、4时

3分钟填写代码,
有问题随时提出!

例1:实现

```
class Solution {
public:
    int searchInsert(std::vector<int>& nums, int target) {
        int index = -1; //最终返回的下标，若找到则为该元素下标，否则为需要插入的位置
        int begin = 0; //搜索区间左端点
        int end = nums.size() - 1; //搜索区间右端点
        while (index == -1) { //若index == -1，则说明还未找到正确位置，持续二分搜索
            int mid = (begin + end) / 2; //计算中心位置
            if (target == nums[mid]) { //若找到target
                index = mid;
            }
            else if (target < nums[mid]) {
                if (mid == 0 || target > nums[mid - 1]) {
                    index = mid;
                }
                end = mid - 1; //如果target小于中点，更新区间右端点
            }
            else if (target > nums[mid]) {
                if (mid == nums.size() - 1 || target < nums[mid + 1]) {
                    index = mid + 1;
                }
                begin = mid + 1; //如果target大于终点，更新区间左端点
            }
        }
        return index;
    }
};
```

位置:[0, 1, 2, 3]

元素:[1, 3, 5, 6]

target = 2、4时

例1:测试与leetcode提交结果

Search Insert Position

Submission Details

62 / 62 test cases passed.

Status: **Accepted**

Runtime: 6 ms

Submitted: 1 hour, 20 minutes ago

i = 0	index = 0
i = 1	index = 0
i = 2	index = 1
i = 3	index = 1
i = 4	index = 2
i = 5	index = 2
i = 6	index = 3
i = 7	index = 4

```
int main() {  
    int test[] = {1, 3, 5, 6};  
    std::vector<int> nums;  
    Solution solve;  
    for (int i = 0; i < 4; i++) {  
        nums.push_back(test[i]);  
    }  
    for (int i = 0; i < 8; i++) {  
        printf("i = %d index = %d\n", i, solve.searchInsert(nums, i));  
    }  
    return 0;  
}
```


例2:区间查找

给定一个**排序数组nums**(nums中有**重复**元素)与**目标值target**，如果target在nums里**出现**，则返回target所在区间的**左右端点下标**，[左端点, 右端点]，如果target在nums里**未出现**，则返回[-1, -1]。

```
class Solution {  
public:  
    std::vector<int> searchRange(std::vector<int>& nums, int target) {  
    }  
};
```

下标	= [0, 1, 2, 3, 4, 5, 6, 7]	target = 8, 返回: [3, 6]
nums	= [5, 7, 7, 8, 8, 8, 8, 10]	target = 6, 返回: [-1, -1]

选自 **LeetCode 34. Search for a Range**

<https://leetcode.com/problems/search-for-a-range/description/>

难度:**Medium**

例2:思考

- 1.可否**直接**通过二分查找，很容易**同时求出**目标target所在区间的**左右端点**？
- 2.若**无法同时求出**区间左右端点，将对目标target的二分查找增加**怎样的限制条件**，就可**分别求出**目标target所在区间的**左端点与右端点**？

下标 = [0, 1, 2, 3, 4, 5, 6, 7]

nums = [5, 7, 7, 8, 8, 8, 8, 10]

target = 8, 返回: [3, 6]

target = 6, 返回: [-1, -1]

例2:算法思路(区间左端点)

查找区间**左端点**时，增加如下**限制条件**：

当 $\text{target} == \text{nums}[\text{mid}]$ 时，若此时 $\text{mid} == 0$ 或 $\text{nums}[\text{mid}-1] < \text{target}$ ，则说明 mid 即为区间左端点，返回；否则设置区间右端点为 $\text{mid}-1$ 。

target = 3

... **1** 3 3 3 3 5 ...



mid 区间左端点

[3 3 3 3 5 ...



mid = 0 区间左端点

... 1 3 **3** 3 3 5 ...



mid



[... 1 3 3]



区间右端点设置为mid - 1

例2:算法思路(区间右端点)

查找区间**右端点**时, 增加如下**限制条件**:

当 $\text{target} == \text{nums}[\text{mid}]$ 时, 若此时 $\text{mid} == \text{nums.size()} - 1$ 或 $\text{nums}[\text{mid} + 1] > \text{target}$, 则说明 mid 即为区间右端点; 否则设置区间左端点为 $\text{mid} + 1$

target = 3

... 1 3 3 3 3 **5** ...



区间右端点 mid

... 1 3 3 3 3 **]**



mid = nums.size() - 1

区间右端点

... 1 3 3 3 3 5 ...



mid



[3 5 ...



区间左端点设置为mid + 1

例2:算法思路

查找区间的右端点:

target = 8;

位置: [0, 1, 2, 3, 4, 5, 6, 7]

元素: [5, 7, 7, 8, 8, 8, 8, 10]



区间右端点

第1次搜索:

[0, 7] [5, 7, 7, 8, 8, 8, 8, 10] mid = 3

nums[mid] == 8, nums[mid+1] == 8,

第2次搜索:

[4, 7] [8, 8, 8, 10] mid = 5

nums[mid] == 8, nums[mid+1] == 8,

第3次搜索:

[6, 7] [8, 10] mid = 6

nums[mid] = 8,

nums[mid+1] == 10 > target

故区间右端点为 6!

例2:课堂练习(区间左端点)

```
int left_bound(std::vector<int>& nums, int target){
    int begin = 0;
    int end = nums.size() - 1;
    while (1){
        int mid = (begin + end) / 2;
        if (target == nums[mid]){
            if (2){
                return mid;
            }
        }
        else if (target < nums[mid]){
            4
        }
        else if (target > nums[mid]){
            5
        }
    }
    return -1;
}
```

3分钟填写代码，
有问题随时提出！

例2:实现(区间左端点)

```
int left_bound(std::vector<int>& nums, int target) {  
    int begin = 0;  
    int end = nums.size() - 1;  
    while (begin <= end) {  
        int mid = (begin + end) / 2;  
        if (target == nums[mid]) {  
            if (mid == nums.size() - 1 || nums[mid + 1] > target) {  
                return mid;  
            }  
            begin = mid + 1;  
        }  
        else if (target < nums[mid]) {  
            end = mid - 1;  
        }  
        else if (target > nums[mid]) {  
            begin = mid + 1;  
        }  
    }  
    return -1;  
}
```

target = 3

... **1** 3 3 3 3 5 ...



mid 区间左端点

[3 3 3 3 5 ...



mid = 0 区间左端点

... 1 3 **3** 3 3 5 ...



mid



[... 1 3 3]



区间右端点设置为mid - 1

例2:课堂练习(区间右端点)

```
int right_bound(std::vector<int>& nums, int target) {  
    int begin = 0;  
    int end = nums.size() - 1;  
    while (begin <= end) {  
        int mid = (begin + end) / 2;  
        if ( 1 ) {  
            if ( 2 ) {  
                return mid;  
            }  
            3  
        }  
        else if ( 4 ) {  
            end = mid - 1;  
        }  
        else if ( 5 ) {  
            begin = mid + 1;  
        }  
    }  
    return -1;  
}
```

3分钟填写代码，
有问题随时提出！

例2:实现(区间右端点)

```
int right_bound(std::vector<int>& nums, int target){
    int begin = 0;
    int end = nums.size() - 1;
    while(begin <= end){
        int mid = (begin + end) / 2;
        if (target == nums[mid]){
            if (mid == nums.size() - 1 || nums[mid + 1] > target){
                return mid;
            }
            begin = mid + 1;
        }
        else if (target < nums[mid]){
            end = mid - 1;
        }
        else if (target > nums[mid]){
            begin = mid + 1;
        }
    }
    return -1;
}
```

target = 3

... 1 3 3 3 3 5 ...



区间右端点 mid

... 1 3 3 3 3]



mid = nums.size() - 1

区间右端点

... 1 3 3 3 3 5 ...



mid



[3 5 ...



区间左端点设置为mid+1

例2:测试与leetcode提交结果

```
class Solution {
public:
    std::vector<int> searchRange(std::vector<int>& nums, int target) {
        std::vector<int> result;
        result.push_back(left_bound(nums, target));
        result.push_back(right_bound(nums, target));
        return result;
    }
};

int main() {
    int test[] = {5, 7, 7, 8, 8, 8, 8, 10};
    std::vector<int> nums;
    Solution solve;
    for (int i = 0; i < 8; i++) {
        nums.push_back(test[i]);
    }
    for (int i = 0; i < 12; i++) {
        std::vector<int> result = solve.searchRange(nums, i);
        printf("%d : [%d, %d]\n", i, result[0], result[1]);
    }
    return 0;
}
```

Search for a Range

Submission Details

87 / 87 test cases passed.

Status: Accepted

Runtime: 16 ms

Submitted: 0 minutes ago

```
0 : [-1, -1]
1 : [-1, -1]
2 : [-1, -1]
3 : [-1, -1]
4 : [-1, -1]
5 : [0, 0]
6 : [-1, -1]
7 : [1, 2]
8 : [3, 6]
9 : [-1, -1]
10 : [7, 7]
11 : [-1, -1]
```



小象学院
ChinaHadoop.cn

例3:旋转数组查找

给定一个**排序数组nums**(nums中有**无重复**元素), 且nums可能以**某个未知**下标**旋转**, 给定**目标值target**, 求target是否在nums中出现, 若出现返回所在下标, 未出现返回-1。

```
class Solution {  
public:  
    int search(std::vector<int>& nums, int target) {  
    }  
};
```

原数组:	[7, 9, 12, 15, 20, 1, 3, 6]
[1, 3, 6, 7, 9, 12, 15, 20]	[9, 12, 15, 20, 1, 3, 6, 7]
可能的旋转结果:	[12, 15, 20, 1, 3, 6, 7, 9]
[3, 6, 7, 9, 12, 15, 20, 1]	[15, 20, 1, 3, 6, 7, 9, 12]
[6, 7, 9, 12, 15, 20, 1, 3]	[20, 1, 3, 6, 7, 9, 12, 15]

选自 **LeetCode 33. Search in Rotated Sorted Array**

<https://leetcode.com/problems/search-in-rotated-sorted-array/description/>

难度:**Medium**

例3:思考

在**旋转数组**[7, 9, 12, 15, 20, 1, 3, 6]中，若**硬使用**未加修改的**二分查找**，查找 $\text{target} = 12$ 或 $\text{target} = 3$ ，会**出现**什么情况？

当前 $\text{mid} = 3$ ， $\text{nums}[\text{mid}] = 15$ ：

查找 $\text{target} = 12$ ：

$\text{target}(12) < \text{nums}[\text{mid}](15)$ ，则在子区间[7, 9, 12]中继续查找，可找到12，返回**正确**结果。

查找 $\text{target} = 3$ ：

$\text{target}(3) < \text{nums}[\text{mid}](15)$ ，则在子区间[7, 9, 12]中继续查找，不可找到3，返回**错误**结果。

思考：

二分查找是否还可以**继续使用**，若希望获得正确的结果，应**如何修改**？

例3:分析

旋转数组:[7, 9, 12, 15, 20, 1, 3, 6], $\text{nums}[\text{begin}] > \text{nums}[\text{end}]$!

查找target = 12 时:

由于 $\text{target}(12) < \text{nums}[\text{mid}](15)$, 查找正确的原因:

1. $\text{nums}[\text{begin}](7) < \text{nums}[\text{mid}](15)$, 区间[7, 9, 12, 15]顺序递增,
2. $\text{target}(12) > \text{nums}[\text{begin}](7)$, 故target(12)只可能在顺序递增区间[7, 9, 12, 15]中。

在查找target = 3 时:

由于 $\text{target}(3) < \text{nums}[\text{mid}](15)$, 查找错误的原因:

1. $\text{nums}[\text{begin}](7) < \text{nums}[\text{mid}](15)$, 区间[20, 1, 3, 6]包括旋转点, 为旋转区间,
2. $\text{target}(3) < \text{nums}[\text{begin}](7)$, 故target(3)可能在旋转区间[20, 1, 3, 6]中, 此时忽略了该情况。

结论:

若希望使用二分查找, 需要修改二分查找, 将可能在旋转区间[20, 1, 3, 6]的情况考虑进去。

例3:算法思路

当目标 $target < \text{中点nums[mid]}$:

如果 $\text{nums[begin]} < \text{nums[mid]}$: (隐含着 $\text{nums[begin]} > \text{nums[end]}$)

(说明递增区间 $[\text{begin}, \text{mid}-1]$, 旋转区间为 $[\text{mid}+1, \text{end}]$)

如: $[7, 9, 12, 15, 20, 1, 3, 6]$

如果 $target(12) \geq \text{nums[begin]}$:

在递增区间 $[\text{begin}, \text{mid}-1]$ 中查找

否则(1):

在旋转区间 $[\text{mid}+1, \text{end}]$ 中查找

如果 $\text{nums[begin]} == \text{nums[mid]}$:

(说明目标只可能在 $[\text{mid}+1, \text{end}]$ 间)

如 $target = 1$, 数组 $[6, 1]$

如果 $\text{nums[begin]} > \text{nums[mid]}$:

(说明递增区间 $[\text{mid}+1, \text{end}]$, 旋转区间为 $[\text{begin}, \text{mid}-1]$)

如: $[20, 1, 3, 6, 7, 9, 12, 15]$

直接在旋转区间 $[\text{begin}, \text{mid}-1]$ 查找

例如3(不可能在比6大的递增区间)

例3:算法思路

当目标 $target >$ 中点 $nums[mid]$:

如果 $nums[begin] < nums[mid]$:

(说明**递增区间** $[begin, mid-1]$, **旋转区间**为 $[mid+1, end]$)

如: $[7, 9, 12, 15, 20, 1, 3, 6]$

直接在**旋转区间** $[mid+1, end]$ 查找

例如查找 $target(20)$

如果 $nums[begin] > nums[mid]$: (隐含着 $nums[begin] > nums[end]$)

(说明**递增区间** $[mid+1, end]$, **旋转区间**为 $[begin, mid-1]$)

如: $[15, 20, 1, 3, 6, 7, 9, 12]$

如果 $target \geq nums[begin]$:

在**旋转区间** $[begin, mid-1]$ 中查找

例如查找20

否则:

在**递增区间** $[mid+1, end]$ 中查找

例如查找9

如果 $nums[begin] == nums[mid]$:

(说明目标**只可能**在 $[mid+1, end]$ 间)

如 $target = 7$, 数组 $[6, 7]$

例3:课堂练习

```
class Solution {
public:
    int search(std::vector<int>& nums, int target) {
        int begin = 0;
        int end = nums.size() - 1;
        while(begin <= end) {
            int mid = (begin + end) / 2;
            if (target == nums[mid]) {
                return mid;
            }
            else if (target < nums[mid]) {
                if (nums[begin] < nums[mid]) {
                    if (1) {
                        end = mid - 1;
                    }
                    else {
                        2
                    }
                }
            }
            else if (nums[begin] > nums[mid]) {
                end = mid - 1;
            }
            else if (nums[begin] == nums[mid]) {
                begin = mid + 1;
            }
        }
    };
};
```

```
else if (target > nums[mid]) {
    if (nums[begin] < nums[mid]) {
        3
    }
    else if (nums[begin] > nums[mid]) {
        if (target >= nums[begin]) {
            4
        }
        else {
            begin = mid + 1;
        }
    }
    else if (nums[begin] == nums[mid]) {
        5
    }
}
return -1;
```

例3:实现1

```
class Solution {
public:
    int search(std::vector<int>& nums, int target) {
        int begin = 0;
        int end = nums.size() - 1;
        while(begin <= end) {
            int mid = (begin + end) / 2;
            if (target == nums[mid]) {
                return mid;
            }
            else if (target < nums[mid]) {
                if (nums[begin] < nums[mid]) {
                    if (target >= nums[begin]) {
                        end = mid - 1;
                    }
                    else {
                        begin = mid + 1;
                    }
                }
                else if (nums[begin] > nums[mid]) {
                    end = mid - 1;
                }
                else if (nums[begin] == nums[mid]) {
                    begin = mid + 1;
                }
            }
        }
    }
}
```

(说明递增区间[begin, mid-1] ,
旋转区间为[mid+1, end])

如:[7, 9, 12, 15, 20, 1, 3, 6]

例如查找7,8,9,10,...,14

例如查找1,2,3,4,5,6

(说明递增区间[mid+1, end] , 旋转区
间为[begin, mid-1])

如:[20, 1, 3, 6, 7, 9, 12, 15]

例如查找1,2,3,4,5,6

(说明目标只可能在[mid+1, end]间)
如target = 1, 数组[6, 1]

例3:实现2

```
else if (target > nums[mid]) {
    if (nums[begin] < nums[mid]) {
        begin = mid + 1;
    }
    else if (nums[begin] > nums[mid]) {
        if (target >= nums[begin]) {
            end = mid - 1;
        }
        else {
            begin = mid + 1;
        }
    }
    else if (nums[begin] == nums[mid]) {
        begin = mid + 1;
    }
}
return -1;
```

(说明递增区间[begin, mid-1] ,
旋转区间为[mid+1, end])

如:[7, 9, 12, 15, 20, 1, 3, 6]

例如查找16,17,18,19,20,21

(说明递增区间[mid+1, end] , 旋转区间为
[begin, mid-1])

如:[15, 20, 1, 3, 6, 7, 9, 12]

例如查找15,16,17,18,...,100,...

如查找4,5,6,7,8,9,10,11,12,13,14

(说明目标只可能在[mid+1, end]间)

如target = 7, 数组[6, 7]

例3:测试与leetcode提交结果

```
int main() {  
    int test[] = {9, 12, 15, 20, 1, 3, 6, 7};  
    std::vector<int> nums;  
    Solution solve;  
    for (int i = 0; i < 8; i++) {  
        nums.push_back(test[i]);  
    }  
    for (int i = 0; i < 22; i++) {  
        printf("%d : %d\n", i, solve.search(nums, i));  
    }  
    return 0;  
}
```

Search in Rotated Sorted Array

Submission Details

196 / 196 test cases passed.

Status: Accepted

Runtime: 6 ms

Submitted: 3 hours, 17 minutes ago

```
0 : -1  
1 : 4  
2 : -1  
3 : 5  
4 : -1  
5 : -1  
6 : 6  
7 : 7  
8 : -1  
9 : 0  
10 : -1  
11 : -1  
12 : 1  
13 : -1  
14 : -1  
15 : 2  
16 : -1  
17 : -1  
18 : -1  
19 : -1  
20 : 3  
21 : -1
```

课间休息10分钟

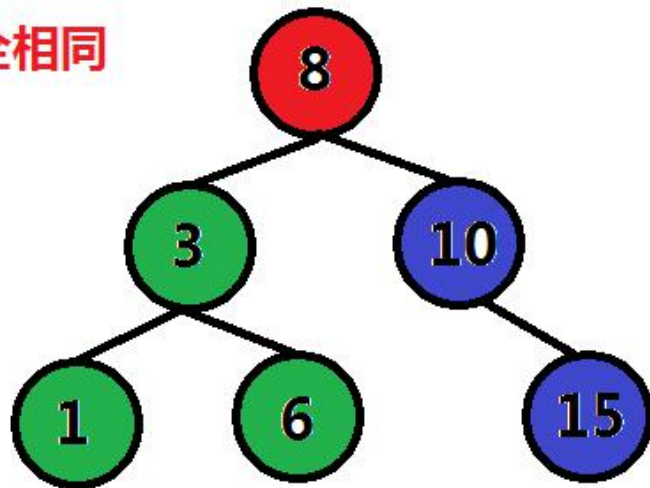
有问题提出！

预备知识:二叉查找(排序)树

二叉查找树(Binary Search Tree), 它是一颗具有下列性质的**二叉树**:

- 1.若**左子树**不空, 则左子树上**所有结点**的值均**小于或等于**它的**根结点**的值;
- 2.若**右子树**不空, 则右子树上**所有结点**的值均**大于或等于**它的**根结点**的值;
- 3.左、右子树也分别为**二叉排序树**。
- 4.**等于**的情况**只能出现在**左子树或右子树中的**某一侧**。

```
struct TreeNode { //与二叉树的数据结构完全相同
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) :
        val(x), left(NULL), right(NULL) {}
};
```



由于二叉查找树的中序遍历是从小到大的, 故又名**二叉排序树**(Binary Sort Tree)。

预备知识:二叉查找树插入节点

将**某节点(insert_node)**, 插入至**以node为根**二叉查找树中:

如果 insert_node节点值 **小于** 当前node节点值:

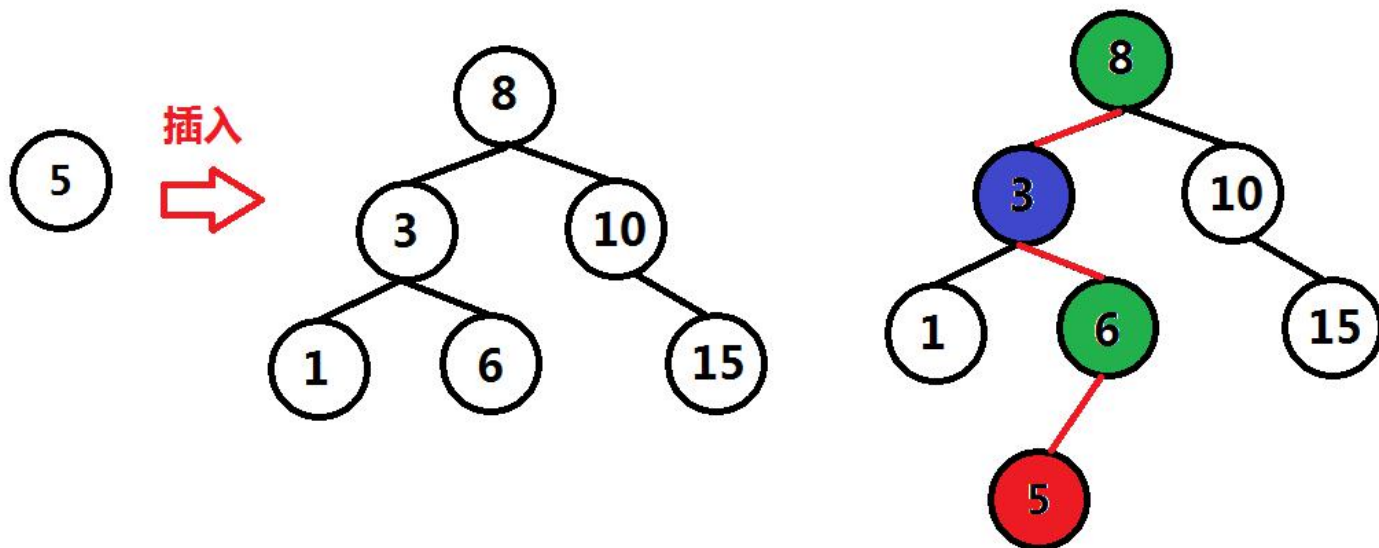
如果node**有左子树**, 则**递归**的将该节点插入至左子树为根二叉排序树中

否则, 将**node->left赋值**为该节点地址

否则(**大于等于**情况):

如果node**有右子树**, 则递归的将该节点插入至右子树为根二叉排序树中

否则, 将**node->right赋值**为该节点地址



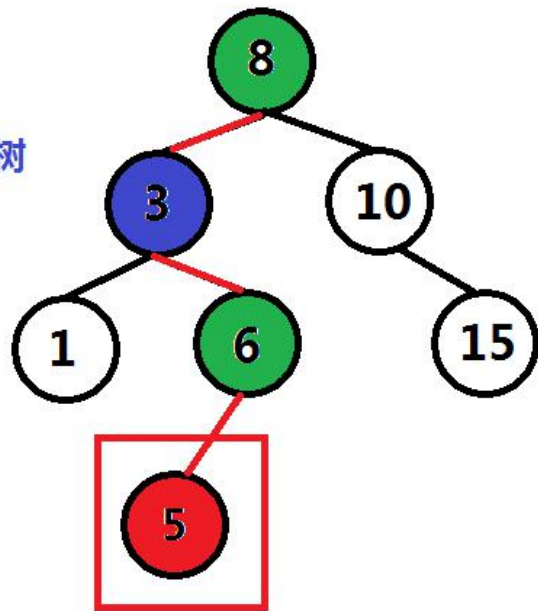
预备知识:课堂练习

```
void BST_insert(TreeNode *node, TreeNode *insert_node) {  
    if ( 1 ) {  
        if ( 2 ) {  
            BST_insert(node->left, insert_node);  
        }  
        else { //当左子树为空时，将node的左指针与待插入节点相连接  
            node->left = insert_node;  
        }  
    }  
    else { //当右子树不空的时候，递归的将insert_node插入右子树  
        if (node->right) {  
            BST_insert(node->right, insert_node);  
        }  
        else {  
            3  
        }  
    }  
}
```

3分钟填写代码，
有问题随时提出！

预备知识:实现

```
void BST_insert(TreeNode *node, TreeNode *insert_node){
    if ( insert_node->val < node->val ) { //当左子树不空的时候，
        if ( node->left ) { //递归的将insert_node插入左子树
            BST_insert(node->left, insert_node);
        }
        else{ //当左子树为空时，将node的左指针与待插入节点相连接
            node->left = insert_node;
        }
    }
    else{ //当右子树不空的时候，递归的将insert_node插入右子树
        if (node->right){
            BST_insert(node->right, insert_node);
        }
        else{ //当右子树为空时，将node的右指针与待插入节点相连接
            node->right = insert_node;
        }
    }
}
```



预备知识:测试

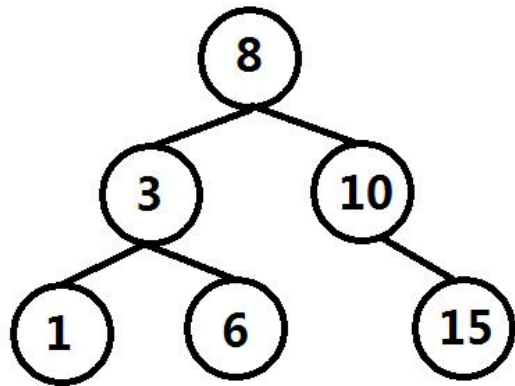
//二叉树前序遍历

```
void preorder_print(TreeNode *node, int layer) {
    if (!node) {
        return;
    }
    for (int i = 0; i < layer; i++) {
        printf("-----");
    }
    printf("[%d]\n", node->val);
    preorder_print(node->left, layer + 1);
    preorder_print(node->right, layer + 1);
}
```

//将test中的节点，按顺序插入到root中

```
int main() {
    TreeNode root(8); //以8为根的二叉树
    std::vector<TreeNode*> node_vec;
    int test[] = {3, 10, 1, 6, 15};
    for (int i = 0; i < 5; i++) {
        node_vec.push_back(new TreeNode(test[i]));
    }
    for (int i = 0; i < node_vec.size(); i++) {
        BST_insert(&root, node_vec[i]);
    }
    preorder_print(&root, 0);
    return 0;
}
```

```
[8]
-----[3]
-----[1]
-----[6]
-----[10]
-----[15]
```



预备知识:二叉查找树查找数值

查找数值value**是否**在**二叉查找树**中**出现**:

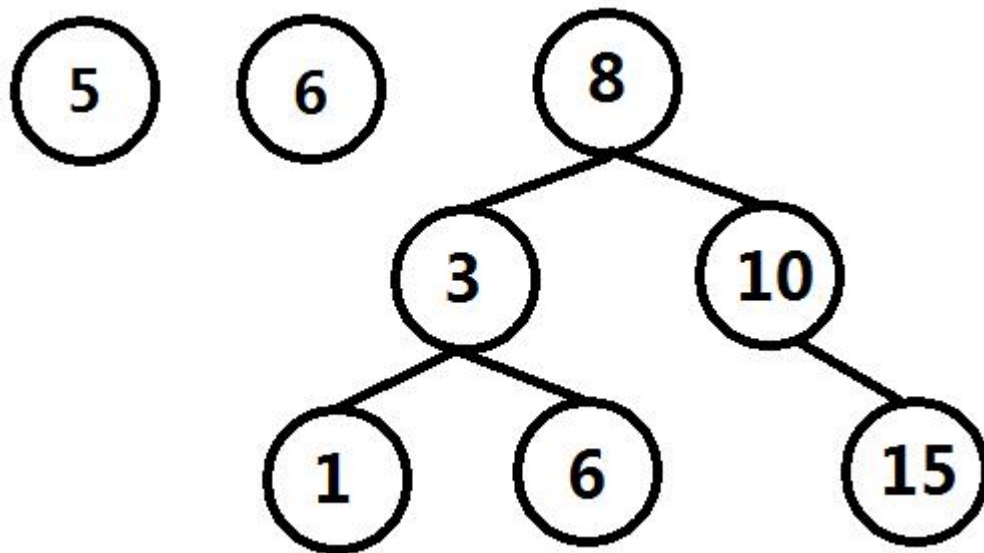
如果 value **等于** 当前查看node的节点值: 返回**真**

如果 value节点值 **小于** 当前node节点值:

如果当前节点**有左子树**, **继续**在**左子树**中查找该值; 否则, 返回**假**

否则(value节点值 **大于** 当前node节点值):

如果当前节点**有右子树**, **继续**在**右子树**中查找该值; 否则, 返回**假**



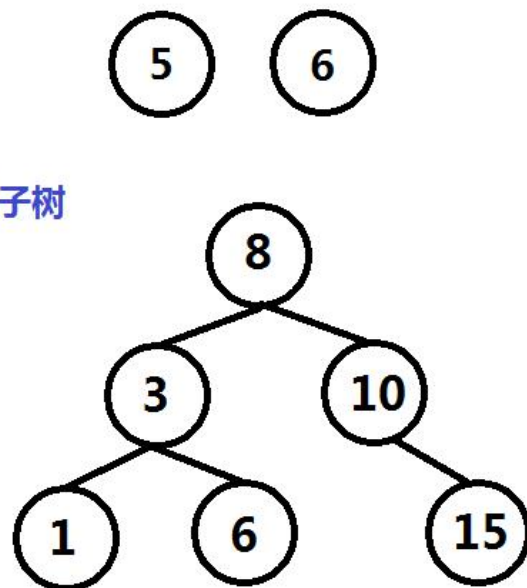
预备知识:课堂练习

```
bool BST_search(TreeNode *node, int value) {  
    if ( 1 ) { ///?? 时 说明找到了  
        return true;  
    }  
    if ( 2 ) { ///?? 时 在左子树中查找  
        if ( 3 ) {  
            return BST_search(node->left, value);  
        }  
        else {  
            return 4  
        }  
    }  
    else { ///右子树不空时  
        if (node->right) {  
            return 5  
        }  
        else {  
            return false;  
        }  
    }  
}
```

3分钟填写代码,
有问题随时提出!

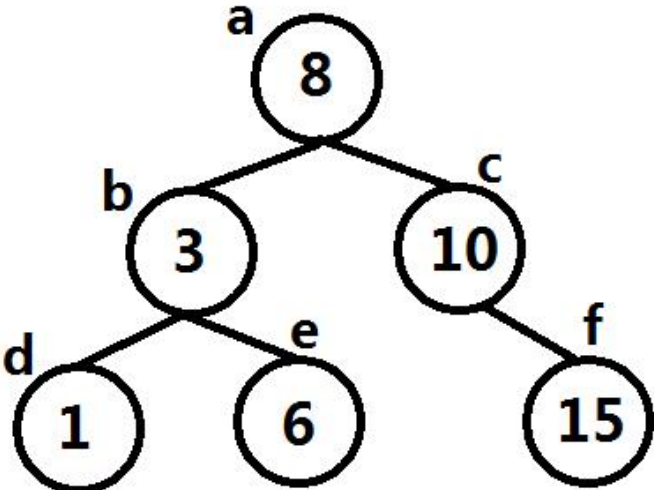
预备知识:实现

```
bool BST_search(TreeNode *node, int value){  
    if (node->val == value) { //当前节点值就是value，找到了返回真  
        return true;  
    }  
    if (value < node->val) { // value值 小于 当前node节点值  
        if (node->left) { //node节点有左子树，继续搜索node节点左子树  
            return BST_search(node->left, value);  
        }  
        else{  
            return false;  
        }  
    }  
    else{  
        if (node->right) { //右子树不空时 继续搜索node节点右子树  
            return BST_search(node->right, value);  
        }  
        else{  
            return false;  
        }  
    }  
}
```



预备知识:测试

```
int main() {
    TreeNode a(8);
    TreeNode b(3);
    TreeNode c(10);
    TreeNode d(1);
    TreeNode e(6);
    TreeNode f(15);
    a.left = &b;
    a.right = &c;
    b.left = &d;
    b.right = &e;
    c.right = &f;
    for (int i = 0; i < 20; i++) {
        if (BST_search(&a, i)) {
            printf("%d is in the BST.\n", i);
        }
        else {
            printf("%d is not in the BST.\n", i);
        }
    }
    return 0;
}
```



```
graph TD
    a((a: 8)) -- left --> b((b: 3))
    a -- right --> c((c: 10))
    b -- left --> d((d: 1))
    b -- right --> e((e: 6))
    c -- right --> f((f: 15))
```

```
0 is not in the BST.
1 is in the BST.
2 is not in the BST.
3 is in the BST.
4 is not in the BST.
5 is not in the BST.
6 is in the BST.
7 is not in the BST.
8 is in the BST.
9 is not in the BST.
10 is in the BST.
11 is not in the BST.
12 is not in the BST.
13 is not in the BST.
14 is not in the BST.
15 is in the BST.
16 is not in the BST.
17 is not in the BST.
18 is not in the BST.
19 is not in the BST.
请按任意键继续. . .
```

例4:二叉查找树编码与解码

给定一个**二叉查找树**，实现对该二叉查找树**编码与解码**功能。编码即将该二叉查找树转为**字符串**，解码即将字符串转为**二叉查找树**。不限制使用何种编码算法，只需**保证**当对二叉查找树调用编码功能后可再调用解码功能将其**复原**。

```
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Codec {
public:
    string serialize(TreeNode* root) {
    }
    TreeNode* deserialize(string data) {
    }
};

// Your Codec object will be instantiated and called as such:
// Codec codec;
// codec.deserialize(codec.serialize(root));
```

选自 **LeetCode 449. Serialize and Deserialize BST**

<https://leetcode.com/problems/serialize-and-deserialize-bst/description/>

难度:**Medium**

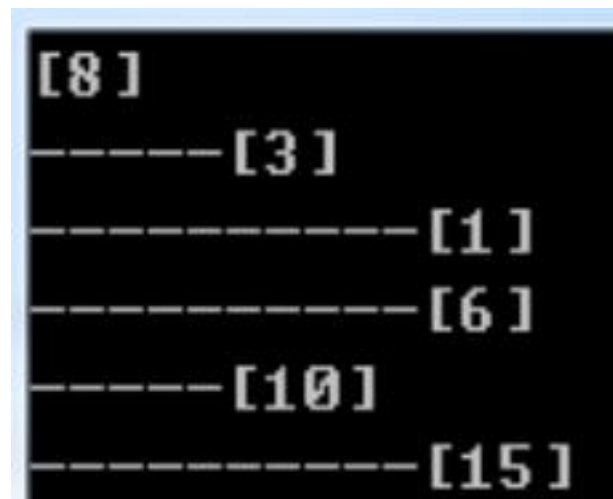
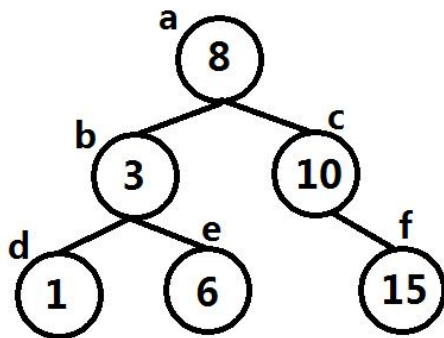
预备知识:二叉查找树前序遍历与复原

对**二叉查找树**进行**前序遍历**，将遍历得到的结果按顺序**重新构造**为一颗**新**的二叉查找树，新的二叉查找树与原二叉查找树**完全一样**。

// 对二叉查找树进行前序遍历，将遍历得到的节点收集到node_vec中

```
void collect_nodes(TreeNode *node, std::vector<TreeNode *> &node_vec) {  
    if (!node) {  
        return;  
    }  
    node_vec.push_back(node);  
    collect_nodes(node->left, node_vec);  
    collect_nodes(node->right, node_vec);  
}
```

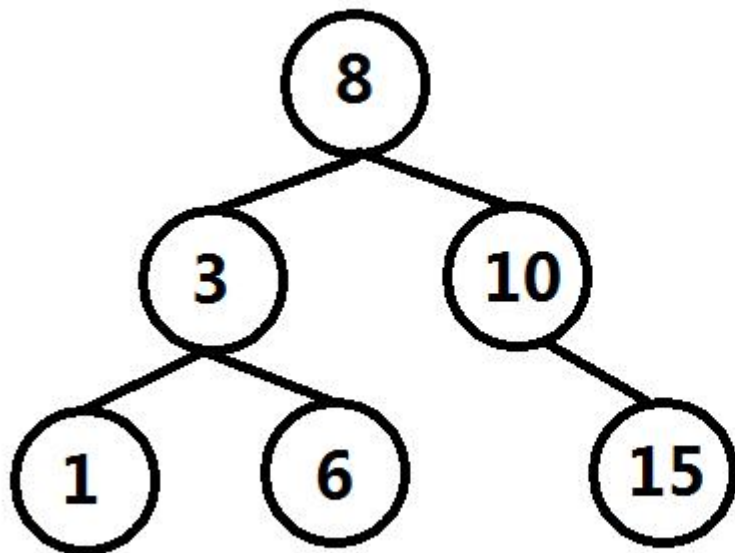
```
int main() {  
    TreeNode a(8);  
    TreeNode b(3);  
    TreeNode c(10);  
    TreeNode d(1);  
    TreeNode e(6);  
    TreeNode f(15);  
    a.left = &b;  
    a.right = &c;  
    b.left = &d;  
    b.right = &e;  
    c.right = &f;  
    std::vector<TreeNode *> node_vec;  
    collect_nodes(&a, node_vec);  
    for (int i = 0; i < node_vec.size(); i++) {  
        node_vec[i]->left = NULL;  
        node_vec[i]->right = NULL;  
    }  
    for (int i = 1; i < node_vec.size(); i++) {  
        BST_insert(node_vec[0], node_vec[i]);  
    }  
    preorder_print(node_vec[0], 0);  
    return 0;  
}
```



例4:算法思路(编码)

二叉查找树**编码**为字符串:

将二叉查找树**前序遍历**，遍历时将**整型的数据转为字符串**，并将这些字符串数据**进行连接**，连接时使用**特殊符号**分隔。



8#

8#3#

8#3#1#

8#3#1#6#

8#3#1#6#10#

8#3#1#6#10#15#

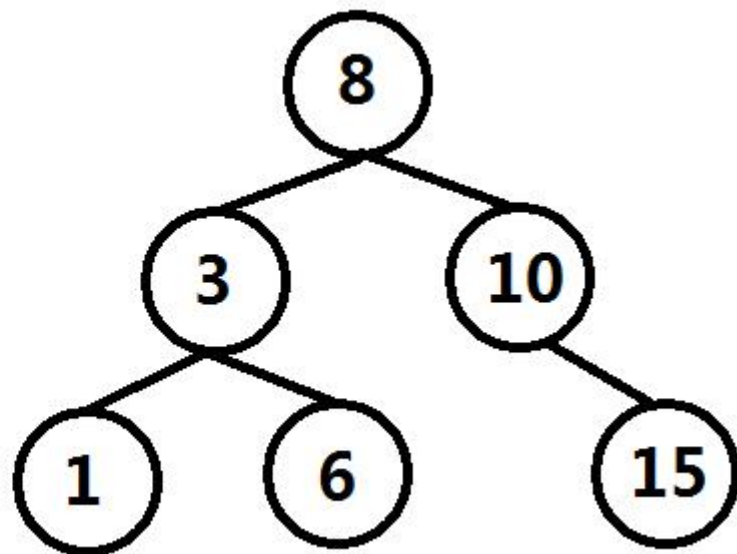
例4:算法思路(解码)

将字符串**解码**为二叉查找树:

将字符串按照编码时的**分隔符”#”**，将各个数字**逐个拆分**出来，将**第一个数字**构建为二叉查找树的**根节点**，后面各个数字构建出的节点按**解析时的顺序**插入根节点中，返回根节点，即完成了**解码**工作。

8#3#1#6#10#15#

根节点



例4:整型转为字符串(编码阶段)

利用对整数10**取余**，可以**逐个循环**的将一个整数**从最低位到最高位**拆分出来，在这个过程中每取出一个最低位，就将该数字除以10。每拆分出一个字符，就将这个字符**添加**到string中，直到该数字为0结束。最后将string进行**反转**，就完成了整数转字符串。

str = ""

num = 12345



'5', str = "5"

num = 1234



'4', str = "54"

num = 123



'3', str = "543"

num = 12



'2', str = "5432"

num = 1



'1', str = "54321"

num = 0

将str反转，变为"12345"，返回。

例4:课堂练习

```
void change_int_to_string(int val, std::string &str_val){
```

```
    std::string tmp;
```

```
    while(val){
```

```
        tmp +=
```

1

//遍历val，每次将val的

最低位转换为字符，添加到tmp的尾部

```
        val = val / 10;
```

```
    }
```

```
    for (int i = tmp.length() - 1; i >= 0; i--){
```

```
        str_val += tmp[i];
```

//逆序将字符串添加到str_val中

```
    }
```

2

```
}
```

```
void BST_preorder(TreeNode *node, std::string &data){
```

```
    if (!node){
```

```
        return;
```

```
    }
```

```
    std::string str_val;
```

```
    change_int_to_string(node->val, str_val);
```

//前序遍历二叉排序树，

将node->val转换为字符串存储至str_val

3

```
    BST_preorder(node->left, data);
```

```
    BST_preorder(node->right, data);
```

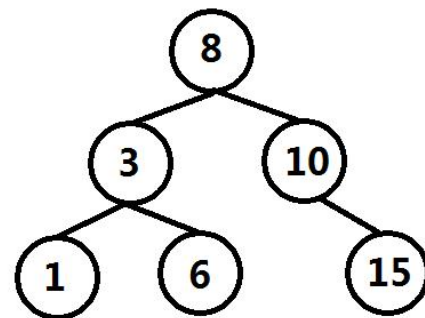
```
}
```

3分钟填写代码，
有问题随时提出！

例4:实现

```
void change_int_to_string(int val, std::string &str_val) {
    std::string tmp;
    while(val) {
        tmp += val % 10 + '0'; //遍历val, 每次将val的
        //最低位转换为字符, 添加到tmp的尾部
        val = val / 10;
    }
    for (int i = tmp.length() - 1; i >= 0; i--) {
        str_val += tmp[i];
    }
    str_val += '#'; //逆序将字符串添加到str_val中
    //转换的每个数字后添加"#"
}

void BST_preorder(TreeNode *node, std::string &data) {
    if (!node) {
        return;
    }
    std::string str_val;
    change_int_to_string(node->val, str_val);
    data += str_val; //前序遍历二叉排序树,
    //将node->val转换为字符串存储至str_val
    //每完成一个整数到字符串的转换,
    //就将该字符串str_val添加到data的尾部
    BST_preorder(node->left, data);
    BST_preorder(node->right, data);
}
```



8#
8#3#
8#3#1#
8#3#1#6#
8#3#1#6#10#
8#3#1#6#10#15#

例4:字符串拆分为整数(解码阶段)

```
#include <stdio.h>
#include <string>

int main() {
    std::string str = "123#456#10000#1#";
    int val = 0;
    for (int i = 0; i < str.length(); i++) {
        if (str[i] == '#') {
            printf("val = %d\n", val);
            val = 0;
        }
        else {
            val = val * 10 + str[i] - '0';
        }
    }
    return 0;
}
```

val = 0
"12345" $\Rightarrow$ **val = 12345**

```
val = 123
val = 456
val = 10000
val = 1
```

"12345"
 $\uparrow$
i = 0 **val = val * 10 + '1' - '0'**
 = 1

"12345"
 $\uparrow$
i = 1 **val = val * 10 + '2' - '0'**
 = 12

"12345"
 $\uparrow$
i = 2 **val = val * 10 + '3' - '0'**
 = 123

"12345"
 $\uparrow$
i = 3 **val = val * 10 + '4' - '0'**
 = 1234

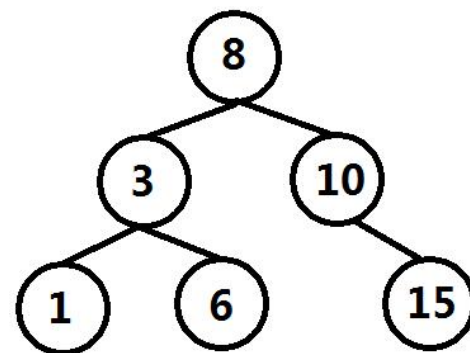
"12345"
 $\uparrow$
i = 4 **val = val * 10 + '5' - '0'**
 = 12345

例4:课堂练习

```
class Codec {
public:
    std::string serialize(TreeNode* root) {
        std::string data;
        BST_preorder(root, data); //将root指向的树转为字符串data
        return data;
    }
    TreeNode* deserialize(std::string data) {
        if (data.length() == 0) {
            return NULL;
        }
        //将提取出的数字，
        //构建二叉查找树节点
        std::vector<TreeNode*> node_vec;
        int val = 0;
        for (int i = 0; i < data.length(); i++) {
            if (data[i] == '#') {
                node_vec.push_back(new TreeNode(val));
                val = 0;
            }
            else {
                1
            }
        }
        for (int i = 1; i < node_vec.size(); i++) {
            BST_insert(node_vec[0], 2);
        }
        3
    }
};
```

8#3#1#6#10#15#

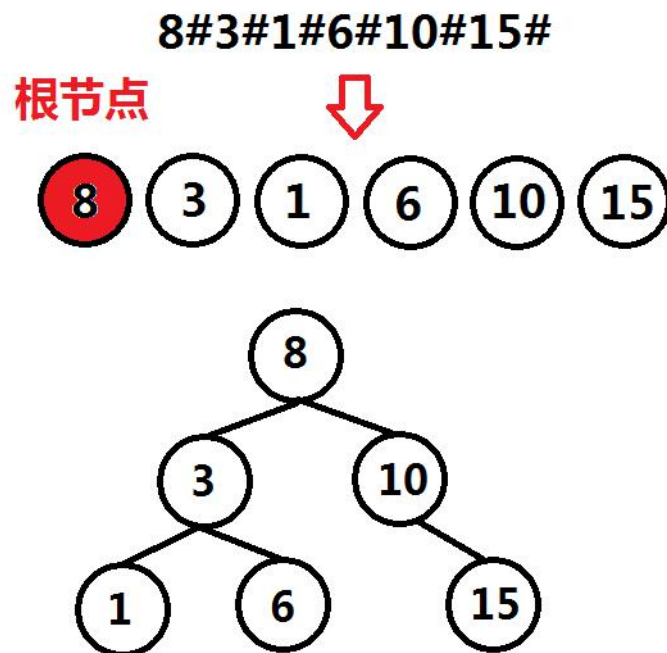
根节点



3分钟填写代码，
有问题随时提出！

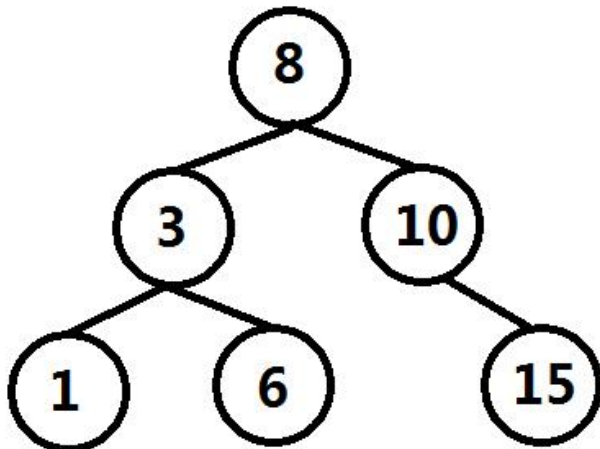
例4:实现

```
class Codec {
public:
    std::string serialize(TreeNode* root) {
        std::string data;
        BST_preorder(root, data); //将root指向的树转为字符串data
        return data;
    }
    TreeNode* deserialize(std::string data) {
        if (data.length() == 0) {
            return NULL;
        }
        //将提取出的数字，构建二叉查找树节点
        std::vector<TreeNode*> node_vec;
        int val = 0;
        for (int i = 0; i < data.length(); i++) {
            if (data[i] == '#') {
                node_vec.push_back(new TreeNode(val));
                val = 0;
            }
            else {
                val = val * 10 + data[i] - '0';
            }
        }
        for (int i = 1; i < node_vec.size(); i++) {
            BST_insert(node_vec[0], node_vec[i]);
        }
        return node_vec[0];
    }
};
```



例4:测试与leetcode提交结果

```
int main() {
    TreeNode a(8);
    TreeNode b(3);
    TreeNode c(10);
    TreeNode d(1);
    TreeNode e(6);
    TreeNode f(15);
    a.left = &b;
    a.right = &c;
    b.left = &d;
    b.right = &e;
    c.left = &f;
    Codec solve;
    std::string data = solve.serialize(&a);
    printf("%s\n", data.c_str());
    TreeNode *root = solve.deserialize(data);
    preorder_print(root, 0);
    return 0;
}
```



```
8#3#1#6#10#15#
[8]
-----[3]
-----[1]
-----[6]
-----[10]
-----[15]
```

Serialize and Deserialize BST

Submission Details

62 / 62 test cases passed.

Status: **Accepted**

Runtime: 23 ms

Submitted: 0 minutes ago

例5:逆序数

已知数组nums，求**新数组count**，count[i]代表了**在nums[i]右侧且比nums[i]小**的元素个数。

例如:

nums = [5, 2, 6, 1], count = [2, 1, 1, 0];

nums = [6, 6, 6, 1, 1, 1], count = [3, 3, 3, 0, 0, 0];

nums = [5, -7, 9, 1, 3, 5, -2, 1], count = [5, 0, 5, 1, 2, 2, 0, 0];

```
class Solution {  
public:  
    std::vector<int> countSmaller(std::vector<int>& nums) {  
    }  
};
```

选自 **LeetCode 315. Count of Smaller Numbers After Self**

<https://leetcode.com/problems/count-of-smaller-numbers-after-self/description/>

难度:**Hard**

例5:思考与分析

已知数组 `nums = [5, -7, 9, 1, 3, 5, -2, 1]`，它的**逆序数**数组 `count = [5, 0, 5, 1, 2, 2, 0, 0]`;

数组 `count[i]`，为 `nums[i+1]`、`nums[i+2]`、...、`nums[size()-1]` 中**有多少个**比 `nums[i+1]`**小**的数；

或者将数组**逆置**`[1, -2, 5, 3, 1, 9, -7, 5]`，

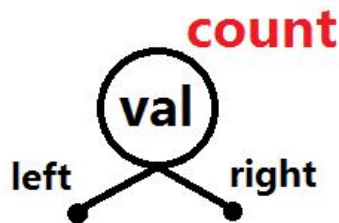
数组 `count[i]` 为 `nums[0]`、`nums[1]`、...、`nums[i-1]` 中**有多少个**比 `nums[i]` 小的数：

- 1, `[]` 中比它小的数个数为0；
- 2, `[1]` 中比它小的数个数为0；
- 5, `[1, -2]` 中比它小的数个数为2；
- 3, `[1, -2, 5]` 中比它小的数个数为2；
- 1, `[1, -2, 5, 3]` 中比它小的数个数为1；
- 9, `[1, -2, 5, 3, 1]` 中比它小的数个数为5；
- 7, `[1, -2, 5, 3, 1, 9]` 中比它小的数个数为0；
- 5, `[1, -2, 5, 3, 1, 9, -7]` 中比它小的数个数为5；

思考：将元素按照**原数组逆置后的顺序**插入到二叉查找树中，如何在元素插入时，**计算**已有多少个元素比当前插入元素小？

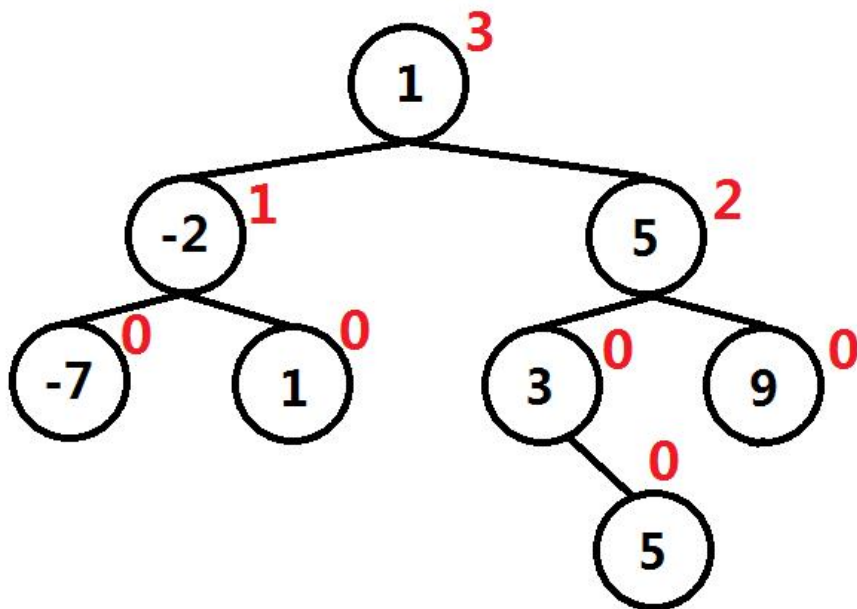
预备知识:记录左子树数量 二叉查找树

```
struct BSTNode {  
    int val;  
    int count;  
    BSTNode *left;  
    BSTNode *right;  
    BSTNode(int x) : val(x),  
        left(NULL), right(NULL), count(0) {}  
};
```



在插入节点时，当待插入节点 **insert_node**
小于等于当前node时 **count++**

按照[1, -2, 5, 3, 1, 9, -7, 5]的顺序构建的二叉查找树



例5:算法思路

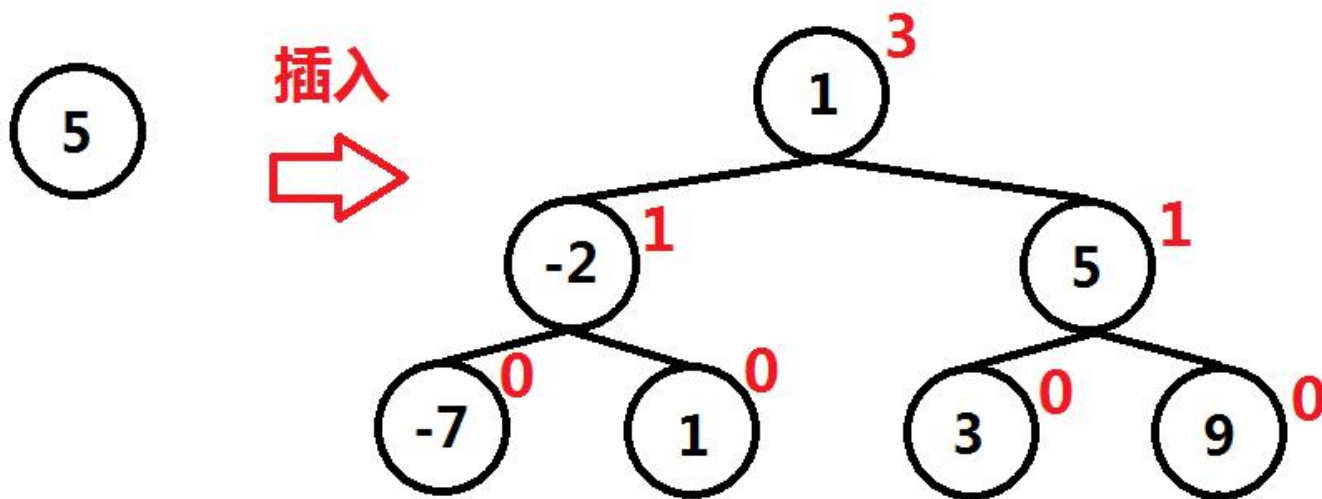
将元素按照**原数组逆置后的顺序**插入到二叉查找树中，如何在元素插入时，**计算**已有多少个元素比当前插入元素小？5，[1, -2, 5, 3, 1, 9, -7]中比它小的数个数为5。

算法如下：

设置变量count_small = 0，记录在插入过程中，有多少个元素比插入节点值小；

若待插入节点值**小于等于**当前节点node值，node->count++，**递归**将该节点插入到当前节点**左子树**；

若待插入节点值**大于**当前节点node值，count_small += node->count + 1(当前节点左子树数量 + 1)；**递归**将该节点插入到当前节点**右子树**。



例5:算法思路

图1: $5 > 1$

`count_small +=`
`node->count + 1`
`count_small = 4`

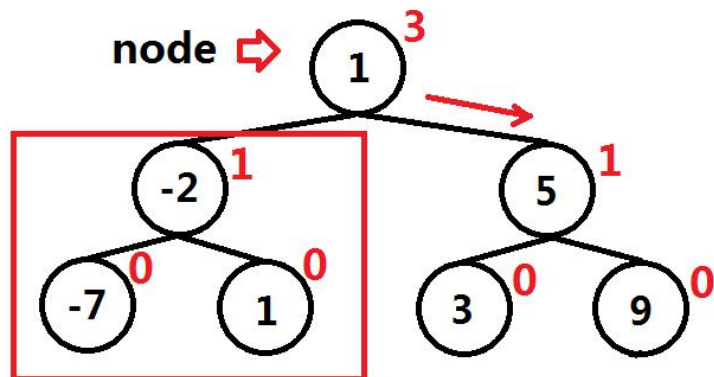


图2: $5 \leq 5$

`node->count ++`

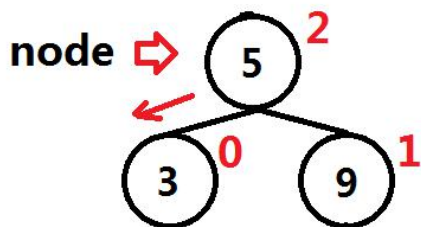
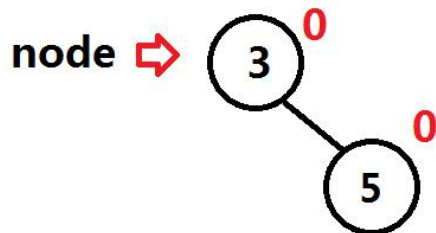
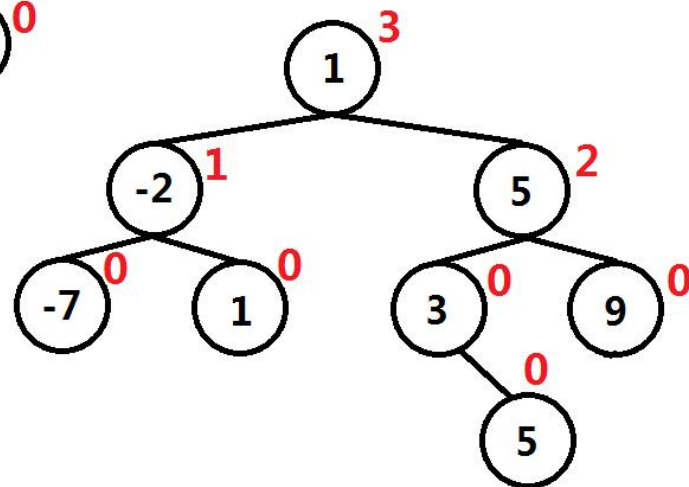


图3: $5 > 3$

`count_small +=`
`node->count + 1`
`count_small = 5`



最终插入5后，二叉查找树树:



例5:课堂练习

```
struct BSTNode {
    int val;
    int count;    //二叉树左子树中节点的个数
    BSTNode *left;
    BSTNode *right;
    BSTNode(int x) : val(x), left(NULL), right(NULL), count(0) {}
};

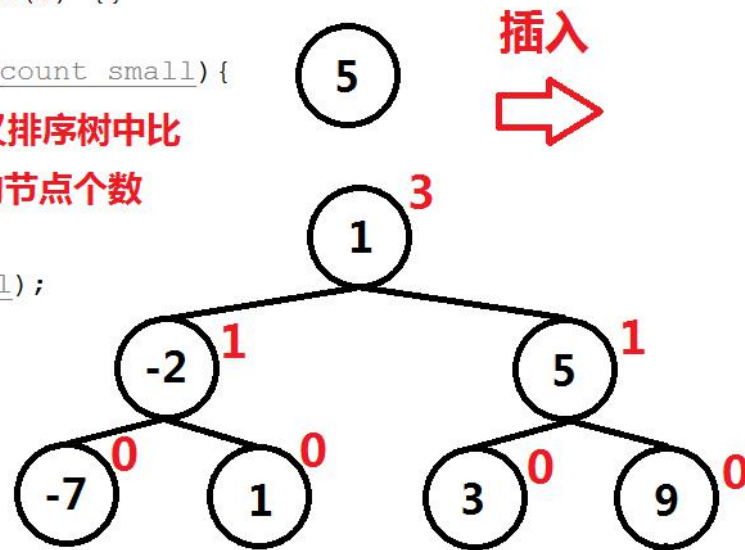
void BST_insert(BSTNode *node, BSTNode *insert_node, int &count_small){
    if (  ) {    //count_small二叉排序树中比
            insert_node值小的节点个数
        if (node->left){
            BST_insert(node->left, insert_node, count_small);
        }
        else{
            node->left = insert_node;
        }
    }
    else{
        
        if (node->right){
            BST_insert(node->right, insert_node, count_small);
        }
        else{
            node->right = insert_node;
        }
    }
}
```

3分钟填写代码，
有问题随时提出！

例5:实现

```
struct BSTNode {
    int val;
    int count;    //二叉树左子树中节点的个数
    BSTNode *left;
    BSTNode *right;
    BSTNode(int x) : val(x), left(NULL), right(NULL), count(0) {}
};

void BST_insert(BSTNode *node, BSTNode *insert_node, int &count_small){
    if (insert_node->val <= node->val) {    //count_small二叉排序树中比
        node->count++;                      insert_node值小的节点个数
    }
    if (node->left) {
        BST_insert(node->left, insert_node, count_small);
    }
    else{
        node->left = insert_node;
    }
}
else{
    count_small += node->count + 1;
    if (node->right) {
        BST_insert(node->right, insert_node, count_small);
    }
    else{
        node->right = insert_node;
    }
}
}
```



例5:整体算法

```
class Solution {
public:
    std::vector<int> countSmaller(std::vector<int>& nums) {
        std::vector<int> result; //最终逆序数数组

        std::vector<BSTNode *> node_vec; //创建的二叉查找树节点池

        std::vector<int> count; //从后向前插入过程中，比当前节点值小的

        for (int i = nums.size() - 1; i >= 0; i--) { count_small数组
            node_vec.push_back(new BSTNode(nums[i]));
        } //按照从后向前的顺序创建二叉查找树节点

        count.push_back(0); //第一个节点count_small = 0

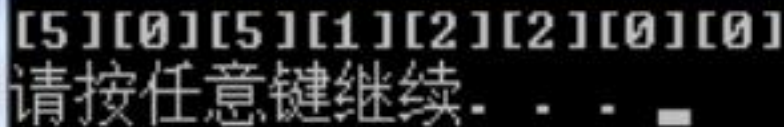
        for (int i = 1; i < node_vec.size(); i++) {
            int count_small = 0;
            BST_insert(node_vec[0], node_vec[i], count_small);
            count.push_back(count_small);
        } //将第2到第n个节点插入到以第1个节点为根的二叉排序中，在插入过程中计算每个节点的count_small

        for (int i = node_vec.size() - 1; i >= 0; i--) { //将count_small数组按照从后向前
            delete node_vec[i];
            result.push_back(count[i]); push进入result数组
        }

        return result; //最终返回result
    }
};
```

例5:测试与leetcode提交结果

```
int main() {  
    int test[] = {5, -7, 9, 1, 3, 5, -2, 1};  
    std::vector<int> nums;  
    for (int i = 0; i < 8; i++) {  
        nums.push_back(test[i]);  
    }  
    Solution solve;  
    std::vector<int> result = solve.countSmaller(nums);  
    for (int i = 0; i < result.size(); i++) {  
        printf("[%d]", result[i]);  
    }  
    printf("\n");  
    return 0;  
}
```



[5][0][5][1][2][2][0][0]
请按任意键继续. . .

Count of Smaller Numbers After Self

Submission Details

16 / 16 test cases passed.

Status: **Accepted**

Runtime: 29 ms

Submitted: 0 minutes ago

结束

非常感谢大家！

林沐